

# PyTorch Tensor Internals — Cheat Sheet

## 1. A Tensor = Storage + Metadata

A tensor is **not just numbers**.

```
Tensor
|
├─ Storage      (actual memory)
├─ Shape        (dimensions)
├─ Stride       (how to move in storage)
├─ Storage Offset (starting position in storage)
├─ Dtype
└─ Device
```

## 2. Storage

- Storage is a **1D contiguous memory block**.
- Storage **doesn't know rows, columns, or dimensions**.
- All dimensions are created using metadata.

Example:

```
Storage

0 1 2 3 4 5 6 7 8 9 10 11
```

## 3. Shape

Shape tells PyTorch **how to interpret storage**.

```
shape = (3,4)
```

↓

```
3 rows
4 columns
```

Changing shape **does not necessarily move data**.

---

## 4. Stride

Stride tells PyTorch:

"How many storage positions should I jump when moving one step along a dimension?"

Example

```
shape = (3,4)

stride = (4,1)
```

Meaning

```
↓ one row    → +4 storage positions
→ one column → +1 storage position
```

Think of stride as **jump sizes**, not formulas.

---

## 5. Storage Offset

Offset tells PyTorch:

"Where does this tensor begin inside storage?"

Example

```
a = torch.arange(10)

b = a[4:]
```

Storage

```
0 1 2 3 4 5 6 7 8 9
      ^
    offset = 4
```

No copy is made.

---

## 6. Walking Through Storage

To reach an element:

- Move along dim0 using stride[0]
- Move along dim1 using stride[1]
- Move along dim2 using stride[2]
- ...

Think of **walking**, not calculating.

---

## 7. View

```
view(new_shape)
```

Meaning:

"Interpret the SAME storage using another shape."

Requirements:

- Total elements must remain the same.
- Tensor must be contiguous.

Changes:

|         |   |
|---------|---|
| Storage | ✗ |
| Shape   | ✓ |
| Stride  | ✓ |
| Offset  | ✗ |

## 8. Permute

```
permute(...)
```

Meaning:

"Reorder dimensions."

Example

```
shape  
(2,3,4)  
  
permute(2,0,1)  
  
↓  
  
(4,2,3)
```

Important:

**Permute BOTH shape and stride.**

```
Shape  
(2,3,4)  
  
↓  
  
(4,2,3)  
  
Stride  
(12,4,1)  
  
↓  
  
(1,12,4)
```

Storage never moves.

---

## 9. Transpose

Special case of permute.

```
transpose(0,2)
```

Only swaps two dimensions.

Storage stays the same.

---

## 10. Contiguous

A tensor is contiguous when its logical order matches the physical memory layout.

`permute()` and `transpose()` usually make tensors **non-contiguous**.

```
view()
```

needs contiguous memory.

If not,

```
tensor.contiguous()
```

creates new storage.

---

## 11. View vs Copy

### View

Shares storage.

```
permute()  
  
transpose()  
  
view()
```

```
slice()  
narrow()  
select()
```

Properties

- Same data\_ptr()
- Same storage
- Changing one changes the other

---

## Copy

Creates new storage.

```
clone()  
contiguous()  
advanced indexing (usually)  
tensor([...])
```

Properties

- Different data\_ptr()
- Independent memory

---

## 12. data\_ptr()

```
a.data_ptr() == b.data_ptr()
```

True

↓

Same storage.

---

## 13. `_base`

```
tensor._base
```

- None → owns its storage
  - Tensor → this tensor is a view
- 

## 14. My Mental Model

Never think:

"What does this function do?"

Always think:

1. Did storage change?
  2. Did shape change?
  3. Did stride change?
  4. Did storage offset change?
  5. Is it contiguous?
  6. Is it a view or a copy?
- 

## 15. Golden Rules

- ✓ Storage = actual memory.
- ✓ Shape = interpretation.
- ✓ Stride = jump sizes.
- ✓ Offset = starting point.
- ✓ View = new metadata, same storage.
- ✓ Copy = new storage.
- ✓ `permute()` reorders **shape and stride**, never storage.
- ✓ `transpose()` is a special case of `permute()`.

✓ `view()` only works on contiguous tensors.

✓ `contiguous()` copies memory only when necessary.

---

## 16. One-Line Summary

**PyTorch tensors are lightweight views over storage. Most tensor operations modify metadata, not memory.**